

Definition of Done

INDICE A / 09 JUILLET 2026

Ce document a pour objectif de définir les critères précis qui doivent être remplis pour considérer qu'une fonctionnalité, une tâche est terminée et prête pour la mise en production. Il s'agit d'un référentiel commun permettant d'assurer la qualité et la cohérence du produit final, en vérifiant que tous les aspects tests, aspects techniques et documentation ont été pris en compte et sont validés.

Critères de tests

Les tests garantissent que chaque fonctionnalité répond aux exigences fonctionnelles et non fonctionnelles. Pour qu'une tâche soit considérée comme « terminée », les critères suivants doivent être validés :

Tests unitaires

Les **tests unitaires** vérifient le comportement isolé des fonctions ou composants du projet (par exemple : un service de calcul de plus-value, un schéma Zod de validation, une méthode de mise à jour du portefeuille virtuel). Garantir que **chaque unité fonctionnelle** (fonction, méthode, service, router tRPC, composant mobile) fonctionne comme prévu, indépendamment des autres composants de l'application.

Ils permettent de détecter rapidement les erreurs introduites lors du développement et facilitent la maintenance en garantissant la non-régression.

- **Runner** : Bun test natif (bun:test) sur tout le monorepo. Les tests sont co-localisés dans `apps/backend/tests/service/*.unit.test.ts` et `apps/backend/tests/router/*.test.ts`.
- **Couverture de test**: Chaque nouvelle fonctionnalité ou modification doit être couverte par des tests unitaires. Le seuil minimum de **90 %** de couverture de lignes est renforcé automatiquement en CI via `coverageThreshold = { line = 0.90 }` dans `apps/backend/bunfig.toml`. Le job CI échoue si la couverture tombe sous ce seuil.

- **Cas de test** : Les tests doivent couvrir les cas d'utilisation normaux, les cas limites et les conditions d'erreur (guards, exceptions tRPC, refus Zod). Chaque service, router ou schéma doit être testé isolément avec ses dépendances mockées.
- **Automatisation** : Les tests unitaires sont automatisés et intégrés dans le processus d'intégration continue. Intégrés dans la CI/CD : toute modification de code (push ou pull request, sur toute branche) déclenche automatiquement l'exécution des tests via GitHub Actions (.github/workflows/ci.yml).
- **Validation obligatoire** : un build ne peut être validé que si tous les tests passent avec succès et que le seuil de couverture 90 % est respecté.

Tests d'intégration

Les **tests d'intégration** valident que les différents modules de l'application (simulation de jeu, gestion de portefeuille, quiz, notifications, etc.) fonctionnent correctement ensemble. Ils permettent de s'assurer que les interactions entre les composants fonctionnent comme prévu, sans erreurs de communication, de format ou de logique métier.

Ils permettent de s'assurer que les modules fonctionnent de manière cohérente en chaîne (par exemple : de la simulation à la mise à jour du portefeuille, puis à l'affichage de récompenses) et qu'un parcours utilisateur réaliste ne produit **ni bugs ni incohérences**.

Exemple : *Simulation d'investissement → Mise à jour du portefeuille → Déverrouillage d'une fonctionnalité* :

L'utilisateur effectue une simulation d'investissement via le Module de Jeu.

Le résultat est transmis au Gestionnaire de Portefeuille pour mise à jour.

Si les conditions de succès sont atteintes (ex. : atteindre 1 000 € de gains simulés), une nouvelle fonctionnalité (comme les crypto-actifs) est déverrouillée.

Interaction des composants :

Module éducatif + quiz : vérifier que, lorsqu'un utilisateur termine un module sur l'épargne, les bonnes questions de quiz sont proposées, et les points sont enregistrés.

Événement aléatoire + portefeuille : après une catastrophe économique simulée, tester que les pertes s'appliquent sur les bons actifs.

Récompenses + interface utilisateur : valider que lorsqu'un objectif est atteint, l'interface affiche correctement la notification et met à jour le profil de l'utilisateur.

Domaines couverts par des tests d'intégration :

- Fin de partie (end-game.service.integration.test.ts) : calcul du résultat final, validation des objectifs, plus-values sur holdings.
- Événements de jeu (game-event-flow.integration.test.ts) : impact d'un événement aléatoire sur les actifs, propagation vers le portefeuille.
- Investissement (investment.service.integration.test.ts) : achat / revente, mise à jour du wallet, calcul des intérêts.
- Objectifs / niveaux (level.service.integration.test.ts, level-goal.service.integration.test.ts, level-event.service.integration.test.ts, goal.service.integration.test.ts).

Exemples de chaînes validées :

- L'utilisateur effectue une transaction via le service investment.
- Le résultat est persisté en base via Prisma dans une transaction atomique.
- Si un objectif de niveau est atteint (ex. wallet_gte_start, min_submarkets_invested), la fin de partie retourne le bon modal (PRIMARY_SUCCESS_ONLY, PRIMARY_AND_SECONDARY_SUCCESS, PRIMARY_FAILURE).

Tests de régression:

Les **tests de régression** permettent de s'assurer que les fonctionnalités existantes continuent de fonctionner comme prévu après l'ajout de nouvelles fonctionnalités, corrections de bugs ou refactorisations.

Ils évitent que des modifications n'introduisent des **effets de bord** non désirés dans des parties stables de l'application.

- **Validation Globale** : Assurer qu'aucune fonctionnalité existante n'est cassée suite à l'ajout ou la modification d'une fonctionnalité.

- Vérifier que les **parcours critiques** de l'application (connexion, progression, simulation, quiz, gestion de portefeuille...) restent **fonctionnels**.
- Détecter toute **cassure ou régression** dans des fonctionnalités déjà validées.
- Préserver la **fiabilité** du système au fur et à mesure de son évolution.

Services actuellement couverts par régression :

- `event.service.regression.test.ts`
- `goal.service.regression.test.ts`
- `level.service.regression.test.ts`

Objectifs :

- Validation globale : assurer qu'aucune fonctionnalité existante n'est cassée suite à l'ajout ou la modification d'une fonctionnalité.
- Détecter toute cassure ou régression dans des fonctionnalités déjà validées (une clause `where`, un `orderBy` ou un `include` modifié par erreur est immédiatement visible dans le diff du snapshot).
- Préserver la fiabilité du système au fur et à mesure de son évolution.

| Critères techniques

Les critères techniques assurent que le code source de l'application est fiable, et les solutions techniques respectent les normes de qualité et les standards de l'équipe de développement.

Qualité du code

Stack imposée :

- **Backend** : Bun + TypeScript strict + tRPC v11 + Prisma + PostgreSQL + Better-Auth.
- **Mobile** : apps/mobile (React Native).

- **Backoffice** : apps/backoffice.
- **Site vitrine** : apps/website.
- **Monorepo** avec apps/* et packages/@cashou/* (packages partagés dont db-app).
- **Normes de codage** : Le code doit impérativement respecter les **conventions définies par l'équipe de développement**, incluant :
 - la **nomenclature des variables, fonctions et composants** (naming convention) doivent être claire, descriptive et cohérente ;
 - une **indentation uniforme** pour assurer la lisibilité (2 espaces) ;
 - une **organisation logique des fichiers et dossiers**, respectant une structure modulaire et évolutive (controllers/ → services/ → trpc/routers/) ;
 - le **respect des règles de syntaxe** imposées par des outils comme ESLint.
- **Analyse Statique** : L'outil d'analyse statique ESLint doit valider le code sans signaler d'erreurs critiques. Le job CI quality exécute bunx eslint . --ext .ts,.tsx et bloque le pipeline en cas d'échec.
- **Revue de Code** : Chaque modification doit être revue par un pair (code review) avant son intégration.
 - Chaque **pull request** doit être validée par au moins un autre développeur.

La revue doit porter sur :

 - La logique métier
 - La lisibilité du code
 - La couverture des tests
 - Les éventuelles régressions possibles

Objectif : **détecter les bugs avant qu'ils n'atteignent la branche principale.**

Architecture et Design

- **Respect de l'architecture** : Toute nouvelle fonctionnalité ou modification doit **s'intégrer harmonieusement** dans l'architecture logicielle existante (ex. : séparation des

responsabilités entre logique métier, interface utilisateur et persistance des données).

- Séparation stricte des couches : trpc/routers/* (contrat API) → trpc/services/* (logique métier) → Prisma (persistance).
- Schémas Zod aux frontières : toute entrée utilisateur est validée par un schéma Zod avant d'atteindre la couche service.
- tRPC comme unique protocole API : typage bout-en-bout entre backend et clients (mobile / backoffice).

Il est impératif de **ne pas introduire de dette technique** (ex. : contournements temporaires, duplication de logique, composants mal placés).

La structure doit être respectée à tous les niveaux : outers tRPC, services métier, modèles Prisma, schémas Zod.

- **Modularité et scalabilité :**

- Le code doit être modulaire, c'est-à-dire organisé en services indépendants, schémas Zod réutilisables et helpers isolés.
- Chaque module (quiz, portefeuille, simulation, récompenses, événements) doit pouvoir être modifié ou enrichi sans impacter les autres.
- Le design doit anticiper une montée en charge ou l'ajout futur de nouveaux types de contenus (nouvelles classes d'actifs, niveaux de difficulté, événements aléatoires, etc.).

Documentation Technique

- **Commentaires et Lisibilité :** Le code doit être commenté de manière à faciliter sa compréhension pour d'autres développeurs.

- Le code doit être auto-descriptif autant que possible (via des noms explicites), et complété par des commentaires pertinents uniquement là où la logique n'est pas immédiatement évidente.
- Les commentaires doivent **expliquer le "pourquoi"**, pas seulement le "quoi".
- Les blocs complexes ou calculs spécifiques (ex. calcul du rendement, conditions de déverrouillage) doivent être **expliqués de manière concise**.
- **Mise à jour des diagrammes** : Les diagrammes d'architecture ou de flux doivent être mis à jour si des modifications significatives ont été apportées.

Toute modification importante de l'**architecture applicative**, de la **navigation**, ou des **flux de données** doit être reflétée dans la documentation visuelle :

| Critères de documentation

Pour qu'une tâche soit considérée comme réellement terminée et livrable, elle doit s'accompagner d'une documentation claire, complète et à jour, à la fois pour les utilisateurs finaux et les équipes techniques.

Documentation utilisateur

- **Guides et Manuels** : Fournir des tutoriels pour les utilisateurs finaux.
 - **Tutoriels interactifs** (dans l'app ou en PDF/vidéo)

Documentation technique

- **Documentation du Code** : Un fichier README et des commentaires explicatifs doivent être fournis pour faciliter la compréhension et la maintenance.

| Conclusion

La Definition Of Done a pour vocation d'assurer que chaque tâche livrée est conforme aux exigences fonctionnelles et techniques, est rigoureusement testée, documentée et prête pour une mise en production sécurisée. Le respect de ces critères garantit une qualité homogène du produit, facilite la maintenance, et assure une transition fluide vers les phases de déploiement.

Ce document devra être revu et mis à jour régulièrement en fonction de l'évolution des pratiques de développement et des exigences du projet. Tous les membres de l'équipe doivent s'y référer afin de garantir une compréhension commune de ce qui constitue une tâche « terminée ».